



HighTech Innovators

YOUR SOFTWARE. BETTER

DevOps++ Sustainability in Your CI/CD Pipeline

**Why is CO₂
a Poor Proxy for
Reducing CO₂?**

Carbon Dioxide

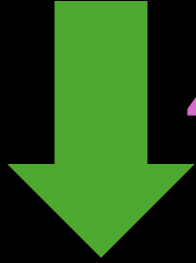
CO₂





41,600,000,000,000,000 grams CO₂e

global CO₂ emissions per year (IPCC/Global Carbon Project)

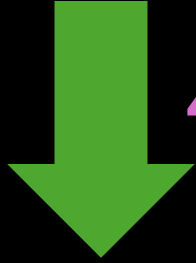


41,600,000,000,000,000 grams CO₂e

global CO₂ emissions per year (IPCC/Global Carbon Project)

46,900,000,000,000,000 grams CO₂e

Scope 3 Rabo customers emissions in 2023



41,600,000,000,000,000 grams CO₂e

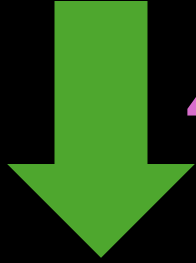
global CO₂ emissions per year (IPCC/Global Carbon Project)

46,900,000,000,000 grams CO₂e

Scope 3 Rabo customers emissions in 2023

69,000,000,000 grams CO₂e

Scope 1 & 2 Rabo operational emissions in 2024



41,600,000,000,000,000 grams CO₂e

global CO₂ emissions per year (IPCC/Global Carbon Project)

46,900,000,000,000 grams CO₂e

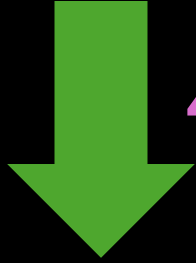
Scope 3 Rabo customers emissions in 2023

69,000,000,000 grams CO₂e

Scope 1 & 2 Rabo operational emissions in 2024

81,030,000 grams CO₂e

1000 virtual example servers (100g co₂e / kWh)



41,600,000,000,000,000 grams CO₂e

global CO₂ emissions per year (IPCC/Global Carbon Project)

46,900,000,000,000 grams CO₂e

Scope 3 Rabo customers emissions in 2023

69,000,000,000 grams CO₂e

Scope 1 & 2 Rabo operational emissions in 2024

81,030,000 grams CO₂e

1000 virtual example servers (100g co₂e / kWh)

220 g CO₂e/kWh

2023 CBS



41,600,000,000,000,000 grams CO₂e

global CO₂ emissions per year (IPCC/Global Carbon Project)

46,900,000,000,000 grams CO₂e

Scope 3 Rabo customers emissions in 2023

69,000,000,000 grams CO₂e

Scope 1 & 2 Rabo operational emissions in 2024

81,030,000 grams CO₂e

1000 virtual example servers (100g co2e / kWh)

220 g CO₂e/kWh

2023 CBS



41,600,000,000,000,000 grams CO₂e

global CO₂ emissions per year (IPCC/Global Carbon Project)

46,900,000,000,000 grams CO₂e

Scope 3 Rabo customers emissions in 2023

69,000,000,000 grams CO₂e

Scope 1 & 2 Rabo operational emissions in 2024

81,030,000 grams CO₂e

1000 virtual example servers (100g co₂e / kWh)

220 g CO₂e/kWh

2023 CBS



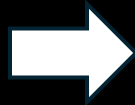
41,600,000,000,000,000 grams CO₂e

global CO₂ emissions per year (IPCC/Global Carbon Project)

46,900,000,000,000 grams CO₂e

Scope 3 Rabo customers emissions in 2023

49.000 employees



69,000,000,000 grams CO₂e

Scope 1 & 2 Rabo operational emissions in 2024

81,030,000 grams CO₂e

1000 virtual example servers (100g co2e / kWh)

220 g CO₂e/kWh

2023 CBS



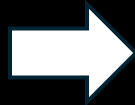
41,600,000,000,000,000 grams CO₂e

global CO₂ emissions per year (IPCC/Global Carbon Project)

12,000,000,000,000 grams CO₂e

Scope 3 ASML emissions in 2024

44.027 employees



32,800,000,000 grams CO₂e

Scope 1 & 2 ASML operational emissions in 2024

81,030,000 grams CO₂e

1000 virtual example servers (100g co2e / kWh)

220 g CO₂e/kWh

2023 CBS



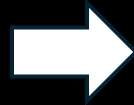
41,600,000,000,000,000 grams CO₂e

global CO₂ emissions per year (IPCC/Global Carbon Project)

813,481,000,000 grams CO₂e

Scope 3 KPN emissions in 2024

10.000 employees



124,564,000,000 grams CO₂e

Scope 1 & 2 KPN operational emissions in 2024

81,030,000 grams CO₂e

1000 virtual example servers (100g co₂e / kWh)

220 g CO₂e/kWh

2023 CBS



41,600,000,000,000,000 grams CO₂e

global CO₂ emissions per year (IPCC/Global Carbon Project)

11,200,000,000,000 grams CO₂e

Scope 3 KLM emissions in 2023

24,758,000,000,000 grams CO₂e

Scope 1 & 2 KLM operational emissions in 2023

81,030,000 grams CO₂e

1000 virtual example servers (100g co₂e / kWh)

220 g CO₂e/kWh

2023 CBS



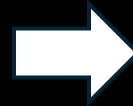
41,600,000,000,000,000 grams CO₂e

global CO₂ emissions per year (IPCC/Global Carbon Project)

163,807,965,000,000 grams CO₂e

Scope 3 Signify emissions in 2024

29.456 employees



204,335,000,000 grams CO₂e

Scope 1 & 2 Signify operational emissions in 2024

81,030,000 grams CO₂e

1000 virtual example servers (100g co2e / kWh)

220 g CO₂e/kWh

2023 CBS



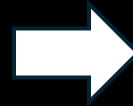
41,600,000,000,000,000 grams CO₂e

global CO₂ emissions per year (IPCC/Global Carbon Project)

163,807,965,000,000 grams CO₂e

Scope 3 Signify emissions in 2024

29.456 employees



204,335,000,000 grams CO₂e

Scope 1 & 2 Signify operational emissions in 2024

405,015,000 grams CO₂e

5000 virtual example servers (100g co2e / kWh)

220 g CO₂e/kWh

2023 CBS

IT/Cloud Cost Overview

Rabobank

IT & software: €644 million, ~7.6% of total operating costs.

ASML

R&D (incl. software): €4.3 billion, ~22–23% of total operation.

KPN

IT & infrastructure: €310 million, ~7.5% of total operating costs.

Estimated VMs (or containers) + CO2

Rabobank

100,000 VMs → 8,103,000,000 grams CO₂e, +11%

ASML

360,000 VMs → 29,171,000,000 grams CO₂e, +90%

KPN

36,000 VMs → 2,917,000,000 grams CO₂e, +2%

1000 Virtual Machines

Monday

Utilization: 50%

Rough CO₂: 439 kg/day

Location: Netherlands

Friday (next week)

Utilization: 75%

Rough CO₂: 222 kg/day

Location: Netherlands

1000 Virtual Machines

Monday

Utilization: 50%

Rough CO₂: 439 kg/day

Location: Netherlands

200 grams CO₂/kWh

Friday (next week)

Utilization: 75%

Rough CO₂: 222 kg/day

Location: Netherlands

100 grams CO₂/kWh



Actions First. CO₂ Second

People connect with energy meters,
switches, routes, bills.

Huge CO₂ numbers feel abstract,
complex, and biased.

Action is tangible → Waste ↓ → CO₂ ↓

CO₂ works best as ***validation***, not as the **driver**.

From DevOps++ to Action

Bringing Sustainability into the CI/CD Pipeline



Once upon a **mission**, a company
was born to change **everything**.

Meet **EcoLogic Unlimited**,
worldwide, 25.000 devs,
3.000+ dev teams,
shipping thousands of apps.

Big ambition:
“We want to be
green & sustainable!”

Plot twist: They forgot **IT**'s impact.

Idle clusters, zombie containers,
unnecessary compute.

Idle clusters, zombie containers,
unnecessary compute.

Huge hidden footprint.

Meet the Sustainable IT Avengers



Greta Volt –
Head of
Sustainability,
vision &
pressure from
the board.

**Sam “CPU-
Slayer” Dev**
– Green
Software
Champion.

Panic Pete
– Ops,
drowning in
servers.

**Aria
Framework**
– Enterprise
Architect,
thinking
long-term.

**Cash
McCloud** –
FinOps
lead, “cloud
bills are
killing me.”

Penny Ledger
– Finance
controller,
tracking
portfolio &
costs.

Why would a Dev Team need DevOps++?

 Even in “perfect” dev teams, we still find waste

Why would a Dev Team need DevOps++?

- ⚡ Fast growth & time-to-market pressure leaves blind spots

Why would a Dev Team need DevOps++?

🙋 Lack of attention / knowledge on sustainable improvements

Why would a Dev Team need DevOps++?

 Idle compute, oversized memory,
over-scaling → always hiding somewhere

We Build DevOps++ on **10 Principles**

Idle Compute Reduction

Ghost energy, monitoring shuts it down.

CPU Load vs Business Value

Not every spike is user demand.

Right-Size Memory

Runtime feedback updates IaC.

IO Visibility

Stop blaming CPUs, fix DB locks.

Validate Early

Perf testing in PRs.

Prune Data

Shrink microservice / serverless payloads.

Compute Location

Edge, client, cloud — choose wisely.

WebAssembly

Lightweight execution.

Performance not by Perception

New nano/millisecond thresholds.

Efficiency in User Terms

OPUA metric (operations per user action).

Sustainability in **IT**
unlocks hidden capacity.

Freeing **people** and **AI**
to solve what matters
together.

Back to EcoLogic Unlimited.
They find **one massive, wasteful app.**

Run the **DevOps++ Playbook**:

1. Traditional AI analytics / Or manual detective work

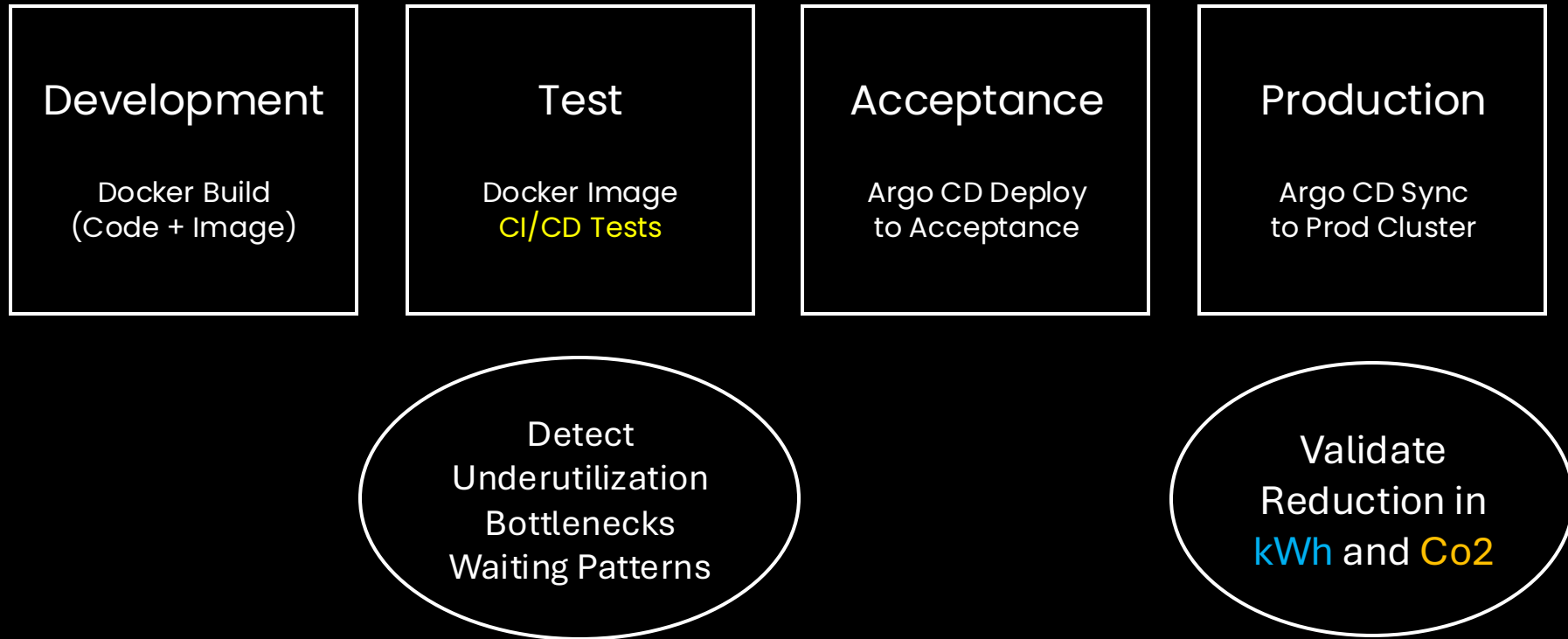
Find top 20 code underutilization, bottlenecks, waiting patterns based on source code & runtime information combined.

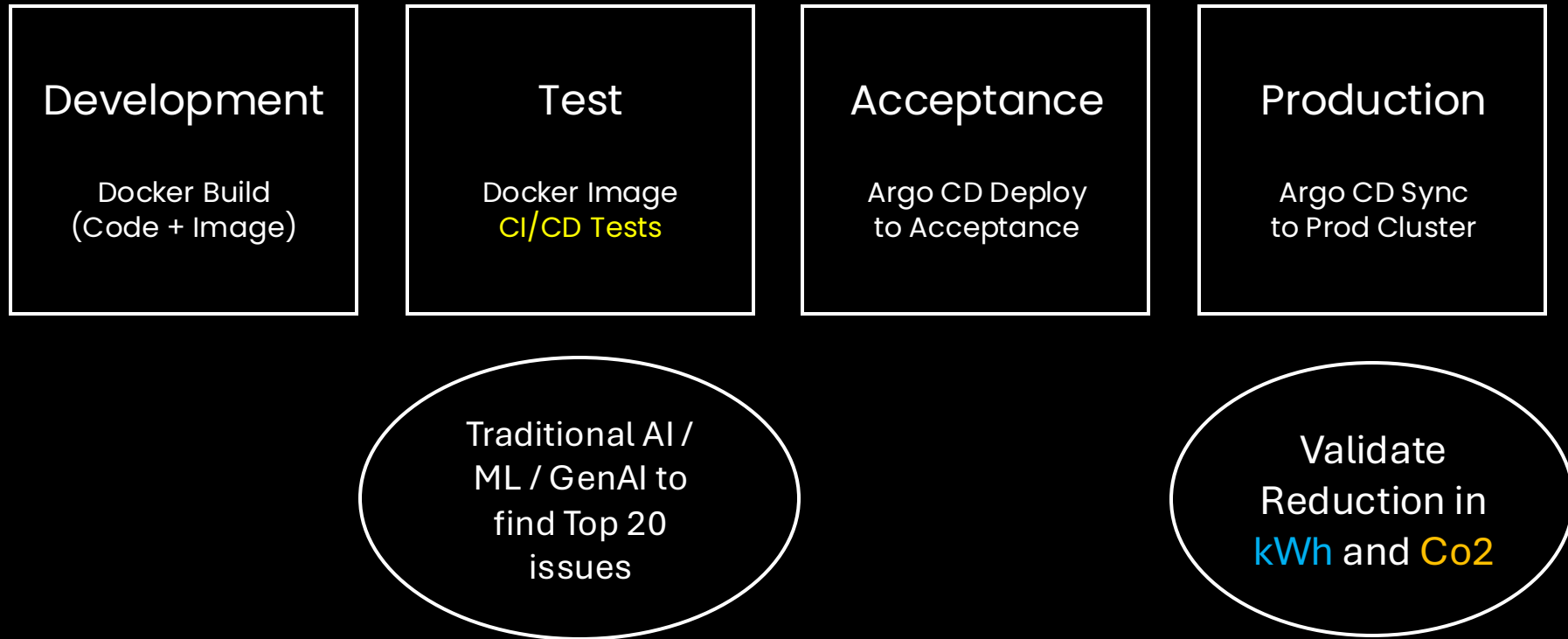
2. Integration tests

Run integration scenario test's that make those underutilization, bottlenecks and waiting patterns visible in running observability also from energy consumption perceptive. And use best tools for this.

3. CI/CD auto-fixes IaC.

4. Ops + FinOps validate in acceptance & prod.





Run the **DevOps++ Playbook**:

1. Traditional AI analytics / Or manual detective work

Find top 20 code underutilization, bottlenecks, waiting patterns based on source code & runtime information.

2. Integration tests

Run integration scenario test's that make those underutilization, bottlenecks and waiting patterns visible in running observability also from energy consumption perceptive. And use best tools for this.

3. CI/CD auto-fixes IaC.

4. Ops + FinOps validate in acceptance & prod.

Aria looks at this...



Core Architectural Responsibilities

Governance

Enforce principles & standards, reuse design patterns.

Reference Models

Govern IaC & scaling, control against baseline.

Alignment

Translate business → IT, balance cost, CO₂, risk.

Future State

Optimize cloud vs. edge, adopt lightweight runtime, define performance thresholds.

And defines these



Run the **DevOps++ Playbook**:

1. Traditional AI analytics / Or manual detective work

Find top 20 code underutilization, bottlenecks, waiting patterns based on source code & runtime information.

2. Integration tests

Run integration scenario test's that make those underutilization, bottlenecks and waiting patterns visible in running observability also from energy consumption perceptive. And use best tools for this.

3. CI/CD auto-fixes IaC.

4. Ops + FinOps validate in acceptance & prod.

Sam is excited.



 Azure DevOps

HTIWorkshops / GreenCoding / Pipelines / Green Coding / 20250130.41

 Search

G

GreenCoding

+

 Overview

 Boards

 Repos

 Pipelines

 Pipelines

 Environments

 Library

 Test Plans

 Artifacts

✓

#20250130.41 • 1mln records

 Green Coding

ⓘ

This run is being retained as one of 3 recent runs by pipeline.

Summary

Code Coverage

Manually run by

Repository and version

 GreenCoding

 matthijs  4e183ab7

Time started and elapsed

 30 Jan at 16:23

 1m 33s

Related

 0 work items

 7 published

Tests and coverage

 Get started

Stages

Jobs

✓ Build .Net Projects

1 job completed

 2 artifacts

17s

✓ Publish Scripting Proj...

1 job completed

5s

✓ Build Java Projects

1 job completed

7s

✓ Deploy to Linux VM

1 job completed

 5 artifacts

40s

← Artifacts

Published

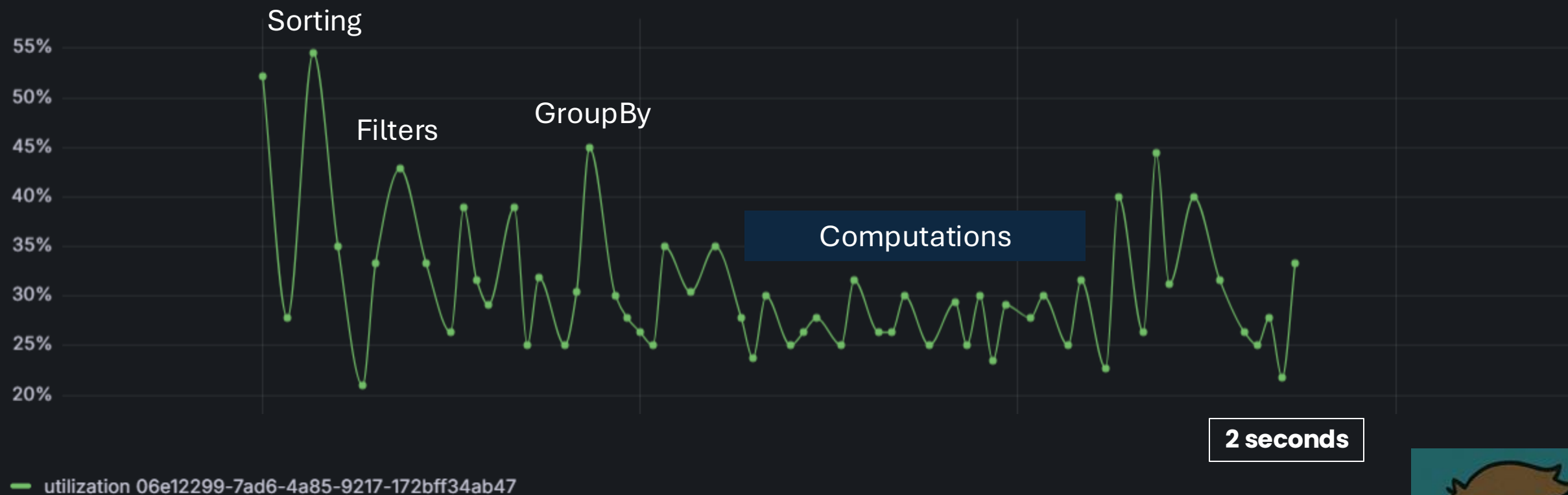
Name	Size	
>  biodiversity-dotnet-artifact	6 MB	
>  biodiversity-java-artifact	56 MB	
>  biodiversity-nodejs-artifact	9 KB	
>  biodiversity-python-artifact	14 KB	
>  CodeTraceLogJava	651 B	
>  EnergyConsumptionReport	176 KB	
>  equal-power-tracer-app-artifact	2 MB	
>  ImpactFrameworkSciOutput	2 KB	
>  Prometheus output	161 KB	⋮
>  SCI Report	3 KB	



<https://localhost/api/GetInfo>

Utilization

Over time



```

1 // ❌ No pruning, no streaming - CPU & memory waste
2 var data = Enumerable.Range(1, 100000)
3     .Select(id => new { Id = id, Value = rnd.Next(1, 10000) })
4     .ToList(); // loads entire dataset up front
5
6 for (int i = 0; i < 50; i++)
7 {
8     var result = data
9         .Where(x => x.Value % 2 == 0)
10        .OrderByDescending(x => x.Value)
11        .GroupBy(x => x.Value % 10)
12        .SelectMany(g => g
13            .GroupBy(x => x.Value % 5)
14            .Select(inner => new
15                {
16                    g.Key,
17                    inner.Key,
18                    OuterAvg = g.Average(y => y.Value),
19                    InnerSum = inner.Sum(y => y.Value)
20                })
21            .ToList(); // materializes huge object graph
22
23     Console.WriteLine(result.Count);
24 }
25

```



```

1 var data = Enumerable.Range(1, 100000)
2   .Select(id => (Id: id, Value: rnd.Next(1, 10000)));
3   // Principle 6: prune data at source
4
5 for (int i = 0; i < 50; i++)
6 {
7     var result = data
8       .AsParallel()
9       // Principle 2: challenge CPU load by controlled parallelism
10      .WithDegreeOfParallelism(Environment.ProcessorCount / 2)
11      .Where(x => (x.Value & 1) == 0)
12      .GroupBy(x => (G10: x.Value % 10, G5: x.Value % 5))
13      .Select(g => new
14      {
15          g.Key.G10,
16          g.Key.G5,
17          Avg = g.Average(y => y.Value),
18          Sum = g.Sum(y => y.Value)
19      })
20      .ToList();
21      // Principle 3: right-size memory - materialize only final result
22
23 Console.WriteLine(result.Count);
24 }

```




```

1 var data = Enumerable.Range(1, 100_000) // ~1 μop/element ~100k μops
2   .Select(id => (Id: id, Value: rnd.Next(1, 10_000))) // ~3 μops/element ~300k μops
3   ; // Principle 6: prune data at source
4
5 for (int i = 0; i < 50; i++) // ~2 μops/iter ~100 μops total
6 {
7     var result = data
8       .AsParallel() // setup ~200 μops one-time
9       .WithDegreeOfParallelism(Environment.ProcessorCount / 2)
10      .Where(x => (x.Value & 1) == 0) // ~1 μop/element ~100k μops
11      .GroupBy(x => (G10: x.Value % 10, G5: x.Value % 5)) // ~15 μops/element ~1.5M μops
12      .Select(g => new
13      {
14          g.Key.G10,
15          g.Key.G5,
16          Avg = g.Average(y => y.Value), // ~3 μops/element ~300k μops
17          Sum = g.Sum(y => y.Value) // ~1 μop/element ~100k μops
18      })
19      .ToList();
20      // Principle 3: right-size memory - materialize only final list (~1 μop/element ~100k μops)
21
22      Console.WriteLine(result.Count); // ~100 μops
23 }
24

```



Utilization

<https://localhost/api/GetInfo>

Over time



1.5 seconds

**Sam is
happy with the
improvements.**



Run the **DevOps++ Playbook**:

1. Traditional AI analytics / Or manual detective work

Find top 20 code underutilization, bottlenecks, waiting patterns based on source code & runtime information.

2. Integration tests

Run integration scenario test's that make those underutilization, bottlenecks and waiting patterns visible in running observability also from energy consumption perceptive. And use best tools for this.

3. CI/CD auto-fixes IaC.

4. Ops + FinOps validate in acceptance & prod.

Pete is skeptical.



Integration Tests

Scenario 1

No orders → function exits idle (P1 ✓).

Scenario 2

Large batch of 100k → async parallel handles faster (P4 ✓).

Scenario 3

Mix of active/inactive orders → only active processed (P2 ✓).



```
3 metadata:
4   name: orders-api
5 spec:
6   replicas: 2 # P1: eliminate idle compute
7   selector: { matchLabels: { app: orders } }
8   template:
9     metadata: { labels: { app: orders } }
10    spec:
11      containers:
12        - name: orders
13          image: registry.example.com/orders:3.2.1
14          resources:
15            requests:
16              cpu: "200m" # P2: challenge CPU load (real telemetry)
17              memory: "256Mi"
18            limits:
19              cpu: "500m"
20              memory: "512Mi" # P3: right-size memory
21          env:
22            - name: MAX_PAYLOAD
23              value: "64k" # P6: prune data at source
```



Run the **DevOps++ Playbook**:

1. Traditional AI analytics / Or manual detective work

Find top 20 code underutilization, bottlenecks, waiting patterns based on source code & runtime information.

2. Integration tests

Run integration scenario test's that make those underutilization, bottlenecks and waiting patterns visible in running observability also from energy consumption perceptive. And use best tools for this.

3. CI/CD auto-fixes IaC.

4. Ops + FinOps validate in acceptance & prod.

Pete remains skeptical, if production breaks, he's the one on the hook, but he's okay with it for now.



Run the **DevOps++ Playbook**:

1. Traditional AI analytics / Or manual detective work

Find top 20 code underutilization, bottlenecks, waiting patterns based on source code & runtime information.

2. Integration tests

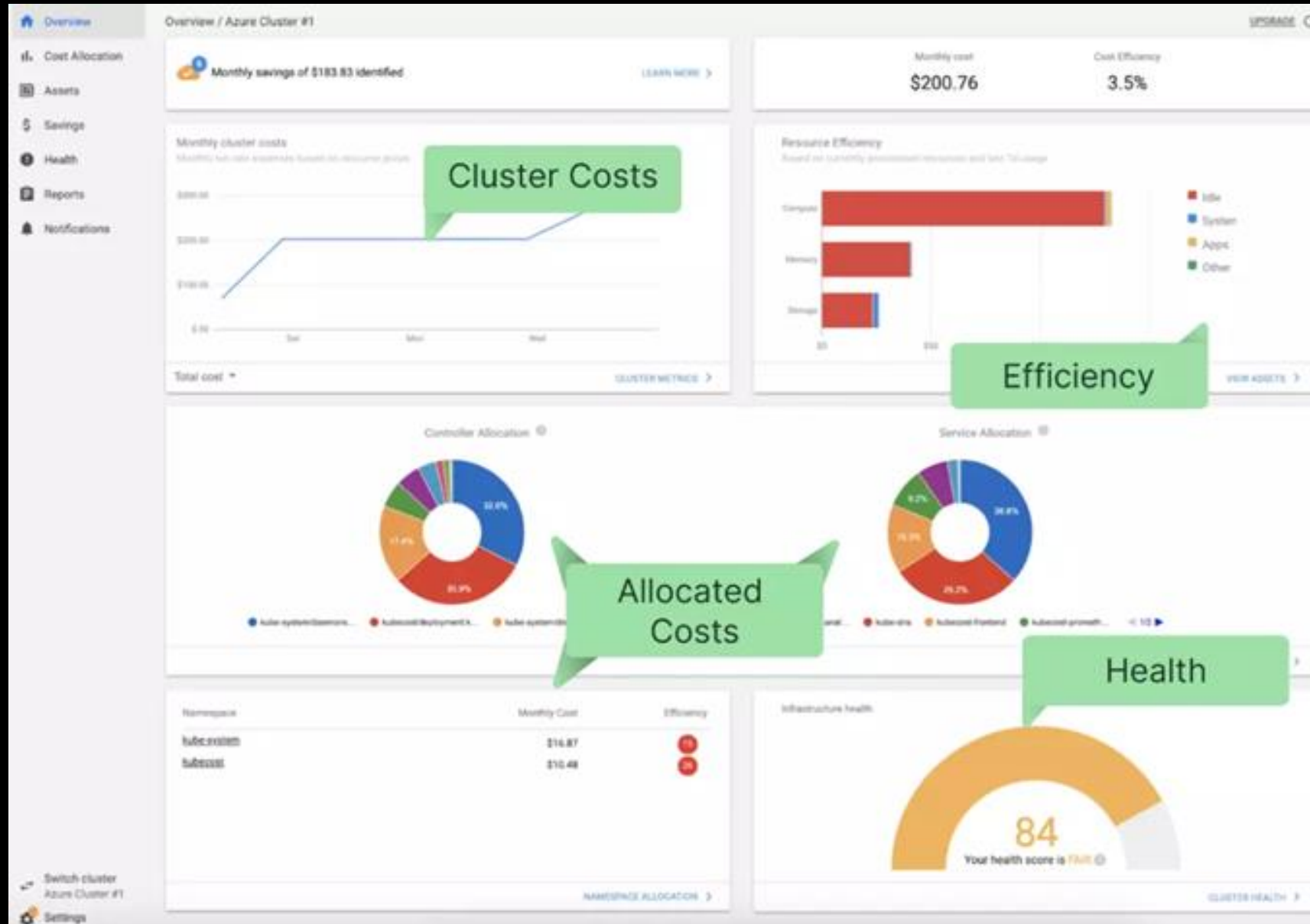
Run integration scenario test's that make those underutilization, bottlenecks and waiting patterns visible in running observability also from energy consumption perceptive. And use best tools for this.

3. CI/CD auto-fixes IaC.

4. Ops + FinOps validate in acceptance & prod.



KubeCost



Cash checks his dashboards



Run the **DevOps++ Playbook**:

1. Traditional AI analytics / Or manual detective work

Find top 20 code underutilization, bottlenecks, waiting patterns based on source code & runtime information.

2. Integration tests

Run integration scenario test's that make those underutilization, bottlenecks and waiting patterns visible in running observability also from energy consumption perceptive. And use best tools for this.

3. CI/CD auto-fixes IaC.

4. Ops + FinOps validate in acceptance & prod.

Cash cheers when cloud bill drops.



Run the **DevOps++ Playbook**:

1. Traditional AI analytics / Or manual detective work

Find top 20 code underutilization, bottlenecks, waiting patterns based on source code & runtime information.

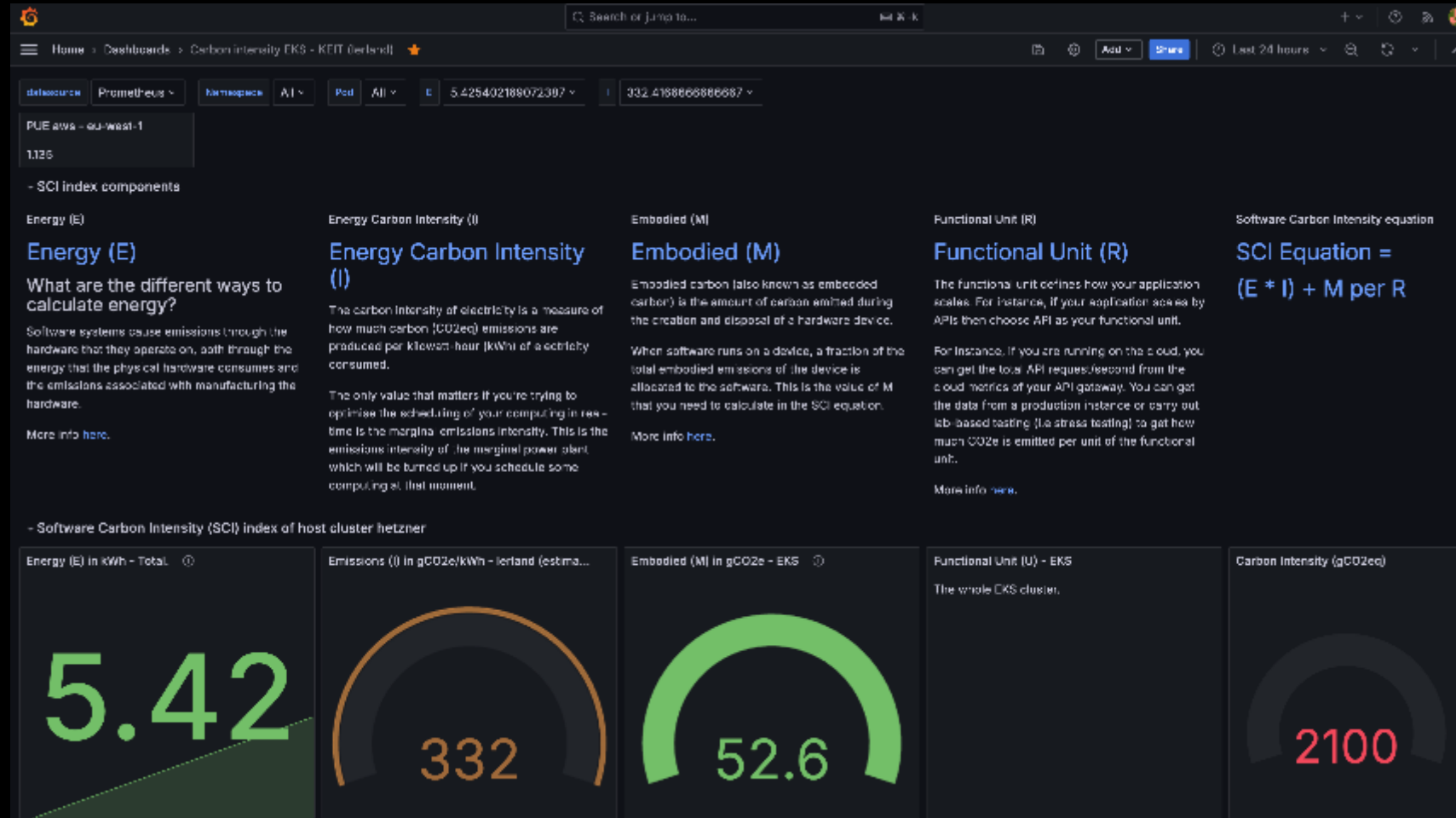
2. Integration tests

Run integration scenario test's that make those underutilization, bottlenecks and waiting patterns visible in running observability also from energy consumption perceptive. And use best tools for this.

3. CI/CD auto-fixes IaC.

4. Ops + FinOps validate in acceptance & prod.





Run the **DevOps++ Playbook**:

1. Traditional AI analytics / Or manual detective work

Find top 20 code underutilization, bottlenecks, waiting patterns based on source code & runtime information.

2. Integration tests

Run integration scenario test's that make those underutilization, bottlenecks and waiting patterns visible in running observability also from energy consumption perceptive. And use best tools for this.

3. CI/CD auto-fixes IaC.

4. Ops + FinOps validate in acceptance & prod.

Greta celebrates CO₂ reduction.



Baseline

☁ 1,000 containers @ 30% utilization

💰 €350k / month

🌍 33.500 kg CO₂ / month

Baseline

- ☁ 1,000 containers @ 30% utilization
- 💰 €350k / month
- 🌍 33.500 kg CO₂ / month



Penny Ledger (Finance Controller) → 😬 "Costs are out of control."

Scenario 1 – Kill Idle

▼ 25% fewer containers

💰 €262k / month

🌍 25.100 kg CO₂ / month

Scenario 1 – Kill Idle

▼ 25% fewer containers

💰 €262k / month

🌍 25.100 kg CO₂ / month



Panic Pete (Ops) → 🗣️ "Finally fewer zombie servers haunting me."

Scenario 2 – Right-Size

▼ 50% less waste

💰 €131k / month

🌍 12.600 kg CO₂ / month

Scenario 2 – Right-Size

▼ 50% less waste

💰 €131k / month

🌍 12.600 kg CO₂ / month



Greta Volt (Head of Sustainability) →
👏 “Massive CO₂ reduction — this is real progress!”

Scenario 3 – Smart Scaling (75% utilization)

 400 containers, same workload

 **€140k / month**

 **33.500 kg CO₂ / month** (unchanged)

Scenario 3 – Smart Scaling (75% utilization)

🚀 400 containers, same workload

💰 €140k / month

🌍 33.500 kg CO₂ / month (unchanged)



Cash McCloud (FinOps Lead) →
🕶️ **"Love the savings... but wait, emissions didn't drop."**

Scenario 4 – Green Workload Shift

-  Workloads moved to **low-carbon regions**
+ renewable datacenters
-  **~€140k / month** (cost stays optimized)
-  **~12.000 kg CO₂ / month** (carbon intensity cut 65%)

Scenario 4 – Green Workload Shift

- 🌍 Workloads moved to **low-carbon regions**
+ renewable datacenters
- 💰 **~€140k / month** (cost stays optimized)
- 🌍 **~12.000 kg CO₂ / month** (carbon intensity cut 65%)



Greta & Cash together → 🤝 “Finally: cost savings and real sustainability wins.”

The DevOps++ pillars

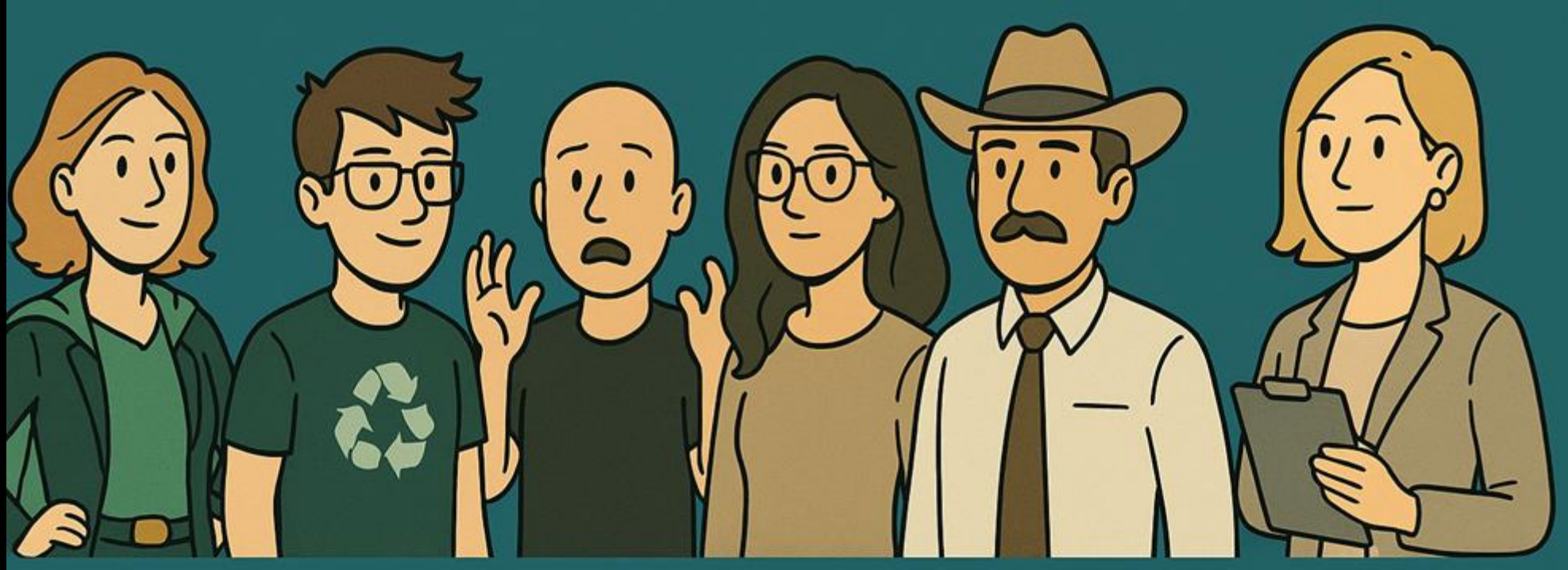
Code
Changes

IaC Changes

Architecture
Repository
Updated

Less
Energy & Co2
In Production

The Sustainable IT Avengers **need help**



Greta Volt –
Head of
Sustainability,
vision &
pressure from
the board.

**Sam “CPU-
Slayer” Dev**
– Green
Software
Champion.

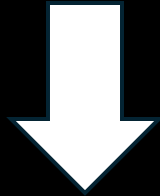
Panic Pete
– Ops,
drowning in
servers.

**Aria
Framework**
– Enterprise
Architect,
thinking
long-term.

**Cash
McCloud** –
FinOps
lead, “cloud
bills are
killing me.”

Penny Ledger
– Finance
controller,
tracking
portfolio &
costs.

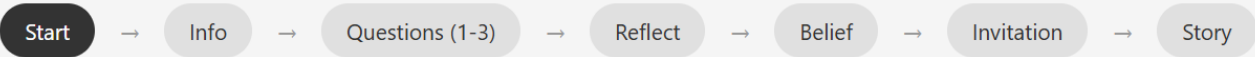
Digital Transformation Architect



Performance Engineer



Clarify App - Interactive Wireframe Flow



ClarifyPreparePresent

Let's start with a few quick questions
We'll tailor the conversation to reveal inefficiencies and opportunities.

Start

ClarifyPreparePresent

Informations

Company name
Acme Corp

Industry
Finance, Retail, Telecom...

Your role
CIO, CFO, VP Engineering...

BackNext →

ClarifyPreparePresent

Diagnostic Question 1/3

How is value versus waste in your IT/AI investment portfolio currently assessed?

A
Standard reporting (cost centers, budget)

B
Delivery/uptime metrics

C
Tie cost to usage/adoption/output

D
Quantify business impact vs IT/AI

BackNext →

ClarifyPreparePresent

Restate & Reflect

Your current state

Multiple teams scale on CPU thresholds; limited cost-value traceability.

ClarifyPreparePresent

Belief Challenge

What evidence ties your current spend to successful business investment?

ClarifyPreparePresent

How We Help Solve This

Based on your inputs, here's a concise walkthrough of our approach.

Restate & Reflect



Your IT/AI portfolio is mostly assessed via budget vs. spend

lorem ipsum dolor.



The main concern seems to be inefficiency in adoption

lorem ipsum dolor.



Your desired outcome is faster alignment between cost and value

lorem ipsum dolor.



Key blockers include lack of clear ownership

lorem ipsum dolor.



Back

Confirm →

Fixing Friction: Ten Example Principles for DevOps++

Wilco Burggraaf · 4 min read · Jul 13, 2025



How Big Companies Can Cut Costs and Carbon Through Smarter Software Operations



In the modern enterprise, DevOps is no longer just about delivering software fast. It's about delivering software that runs efficiently, scales with purpose, and emits as little waste — digital, financial, and environmental — as possible. When you step inside large organizations with sprawling Kubernetes clusters, hybrid clouds, and thousands of containerized workloads, the patterns of waste are not always visible. But they are absolutely measurable. They sit in logs, dashboards, forgotten queues, and over-provisioned clusters.

This article lays out ten working principles — living examples that guide the evolving DevOps++ approach. These are not commandments or fixed rules. They are practical foundations, shaped by experience, and intended to be iterated upon by the community. DevOps++ itself is not a product, not a certification, and not a proprietary framework. It's an open, shared language for building software systems that are efficient, transparent, and resilient. It integrates development, infrastructure-as-code, and architecture into a unified rhythm where every change is an opportunity to lower cost, reduce footprint, and create more durable value.

Principle 1: Eliminate Idle Compute

Idle compute remains one of the most neglected drains in large environments. Containers spin in staging, dev clusters go forgotten, and workloads stay warm long past their use. DevOps++ bakes runtime activity checks into CI/CD workflows, ensuring policies are triggered to downscale or shut down compute when it no longer generates value. This is not about setting timers. It's about using application signals to guide compute lifecycles.

Principle 2: Challenge CPU Load Without Business Context

High CPU usage has long been mistaken as a symptom of scale. But not all scale is business value. In a DevOps++ world, teams trace CPU utilization back to business logic. Was that spike user-driven? Was it caused by inefficient polling? Every autoscaling policy is reviewed not by thresholds alone, but by relevance to product and impact on users.

Principle 3: Right-Size Memory With Runtime Feedback

No team gets memory sizing right the first time. That's why DevOps++ encourages memory observability to be integrated into CI test runs. Teams use historical runtime traces to automatically update IaC files with recommended limits. The result is fewer OOM crashes and far less memory waste.

Principle 4: Prioritize IO Visibility Over Compute Assumptions

Slow systems are often misunderstood. More cores are added, only to find the issue lives in disk IO, database locks, or blocked sockets. DevOps++ insists on full-stack latency tracing embedded into every deployment flow. Bottlenecks are located where they are — not where dashboards are easiest to read.

